Name: V. Alok
H. No : 2303A53003

SCHOOL OF COMPUTER SCIENCE AND ARTIFICIAL INTELLIGENCE		DEPARTMENT OF COMPUTER SCIENCE ENGINEERING	
Program Name: B. Tech		**Assignment Type: Lab**	**Academic Year:**2025-2026
Course Coordinator Name	Dr. Rishabh Mittal		
Instructor(s) Name	Mr. S Naresh Kumar Ms. B. Swathi Dr. Sasanko Shekhar Gantayat Mr. Md Sallauddin Dr. Mathivanan Mr. Y Srikanth Ms. N Shilpa Dr. Rishabh Mittal (Coordinator) Dr. R. Prashant Kumar Mr. Ankushavali MD Mr. B Viswanath Ms. Sujitha Reddy Ms. A. Anitha Ms. M.Madhuri Ms. Katherashala Swetha Ms. Velpula sumalatha Mr. Bingi Raju		
CourseCode	23CS002PC304	**Course Title**	AI Assisted Coding
Year/Sem	III/II	**Regulation**	R23
Date and Day of Assignment	**Week7 – Wednusday**	**Time(s)**	23CSBTB01 To 23CSBTB52
Duration	2 Hours	**Applicable to Batches**	All batches
Assignment Number:13.3(Present assignment number)/**24**(Total number of assignments)			

Q.No.	Question	*Expected Time to complete*

	Lab 13: Code Refactoring Using AI Assistance Improving Legacy Code for Readability, Maintainability, and Performance **Lab Objectives** The objectives of this laboratory exercise are to: • Identify common code smells and inefficiencies in legacy Python programs. • Use AI-assisted coding tools to refactor code for better readability and maintainability. • Apply modern Python best practices while preserving original program behavior. • Analyze performance improvements achieved through refactoring.	
	Learning Outcomes After completing this lab, students will be able to: • Detect and eliminate code duplication in Python programs. • Refactor inefficient logic using optimized data structures and constructs. • Improve code quality through meaningful naming conventions and documentation. • Validate refactored code using test cases and performance comparison.	

1	**Task 1: Refactoring – Removing Code Duplication**
	Objective
	To eliminate repeated logic by extracting reusable functions.
	Task Description
	Use AI assistance to refactor a legacy Python script that contains repeated blocks of code calculating the area and perimeter of rectangles.
	Starter (Legacy) Code
	# Legacy script with repeated logic
	print("Area of Rectangle:", 5 * 10)
	print("Perimeter of Rectangle:", 2 * (5 + 10))
	print("Area of Rectangle:", 7 * 12)
	print("Perimeter of Rectangle:", 2 * (7 + 12))
	print("Area of Rectangle:", 10 * 15)
	print("Perimeter of Rectangle:", 2 * (10 + 15))
	Expected Outcome
	• A reusable function to calculate area and perimeter
	• No duplicated code blocks
	• Proper docstrings for all functions
	
	Prompt:
	
	Refactor the following legacy Python code by removing duplicated logic.
	
	Requirements:
	- Create a reusable function to calculate area and perimeter of a rectangle.
	- Use proper function naming.
	- Add clear docstrings.
	- Ensure output remains the same.
	- Follow modern Python best practices.
		Week7 - Wednesday

Legacy code:

```
print("Area of Rectangle:", 5 * 10)
print("Perimeter of Rectangle:", 2 * (5 + 10))
print("Area of Rectangle:", 7 * 12)
print("Perimeter of Rectangle:", 2 * (7 + 12))
print("Area of Rectangle:", 10 * 15)
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

Task 2: Refactoring – Optimizing Loops and Conditionals

Objective

To improve performance by replacing inefficient nested loops with optimized structures.

Task Description

Use AI to analyze and refactor a script that checks the presence of elements using nested loops.

Starter (Legacy) Code

```
names = ["Alice", "Bob", "Charlie", "David"]
search_names = ["Charlie", "Eve", "Bob"]
for s in search_names:
found = False
for n in names:
if s == n:
found = True
if found:
print(f"{s} is in the list")
else:
print(f"{s} is not in the list")
```

Expected Outcome

- Optimized solution using set lookups or comprehensions
- Performance comparison before and after refactoring

Prompt:

Analyze and refactor this Python code to improve performance.

Requirements:
- Replace inefficient nested loops with optimized logic.
- Use sets or efficient lookup structures.
- Compare performance before and after refactoring.
- Keep output unchanged.
- Explain why the new version is faster.

Legacy code:

```
names = ["Alice", "Bob", "Charlie", "David"]
```

```
search_names = ["Charlie", "Eve", "Bob"]

for s in search_names:
    found = False
    for n in names:
        if s == n:
            found = True
    if found:
        print(f"{s} is in the list")
    else:
        print(f"{s} is not in the list")
```

Task 3: Refactoring – Extracting Reusable Functions
Objective
To modularize code by extracting calculations into reusable functions.
Task Description
Refactor a legacy script where price and tax calculations are written inline.
Starter (Legacy) Code

```
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)

price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

Expected Outcome

- A function calculate_total(price)
- Cleaner and reusable code structure
- Proper documentation

Prompt:

Refactor this legacy Python script by extracting reusable functions.

Requirements:
- Create a function named calculate_total(price).
- Include tax calculation inside the function.
- Add proper docstrings.
- Improve readability and reusability.
- Maintain identical output.

Legacy code:

```
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

```
price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

Task 4: Refactoring – Replacing Hardcoded Values with Constants
Objective
To improve maintainability by replacing magic numbers with named constants.
Task Description
Use AI to identify hardcoded values and replace them with constants.
Starter (Legacy) Code

```
print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)
```

Expected Outcome

- Constants such as PI and RADIUS
- Cleaner, maintainable code

Prompt:

Refactor this Python code by replacing magic numbers with named constants.

Requirements:
- Use constants such as PI and RADIUS.
- Improve readability and maintainability.
- Keep logic and output unchanged.
- Follow Python constant naming conventions.

Legacy code:

```
print("Area of Circle:", 3.14159 * (7 ** 2))
print("Circumference of Circle:", 2 * 3.14159 * 7)
```

Task 5: Refactoring – Improving Variable Naming and Readability
Objective
To enhance readability using descriptive variable names and comments.
Task Description
Refactor a script with unclear variable names.
Starter (Legacy) Code

```
a = 10
b = 20
c = a * b / 2
print(c)
```

Expected Outcome

- Descriptive variable names
- Inline comments explaining logic
- Identical output

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

	Prompt: Improve the readability of this Python code. Requirements: - Replace unclear variable names with descriptive ones. - Add inline comments explaining the logic. - Keep output exactly the same. - Follow clean code principles. **Legacy code:** a = 10 b = 20 c = a * b / 2 print(c)	